牙齿模型功能技术路线前端视角

基于现有展示 边界编辑与牙根模拟效果的参考思路

从前端角度看，可以沿用现有 Vue 和 Three.js 的实现思路：把牙齿数据转成可交互的三维场景，操作时及时预览，再把明确的编辑结果保存下来。需要复杂重算的部分，与后台和算法约定后接入。

一 前端整体怎么做

获取模型数据 → 解析并生成三维场景 → 选牙和编辑
→ 实时预览 → 提交保存或请求重算 → 加载结果并恢复显示

现有 msgpackTeeth 页面侧重模型分区与边界编辑，toothRoots 页面侧重牙根模拟、单牙姿态和咬合展示。可以在此基础上复用数据读取、场景交互和保存恢复的能力。

二 现有效果在前端怎样实现

效果	前端实现思路
展示上下颌和单牙	读取 MessagePack 数据，用 Draco 解压模型；根据牙号标签生成不同颜色的三维网格。
选择和查看牙齿	用射线拾取定位点击对象；提供选牙、显隐、旋转视角、缩放和平移。
拖动牙齿边界	将控制点和轮廓贴合模型表面，拖动时更新边界；符合条件时提交局部标签变化。
模拟牙根与调整牙齿	根据牙颈和方向拟合根模板；把牙冠和根放在同一组中，统一移动和旋转。
咬合和分析提示	调整下颌整体姿态，使用三维几何查询显示碰撞、间隙和截面。

三 前端内部建议分成三层

数据层：保留原始模型、牙号、边界和方向，统一处理数据格式。显示时需要复制或拆分网格，不能把显示副本直接当成原始数据。

交互层：管理选中牙齿、控制点、姿态和编辑状态。拖动时优先保证反馈及时，松手后再校验并提交；失败时恢复到可用状态。

显示层：负责牙冠、牙根、边界和分析标记的渲染。显示结果应由数据和编辑状态生成，避免“界面看起来改了，保存的数据却没改”。

四 前端需要怎样与后台和算法配合

向后台要什么

能获取同一病例指定版本的完整模型数据；能提交并保存编辑结果；需要算法重算时，能发起任务并取得结果或失败原因。后台应返回明确的数据版本，前端才能确认重新打开的是哪一次编辑。

前端提交什么要先约定：边界编辑提交牙号和边界或标签变化，姿态编辑提交位置、旋转和牙根参数。可以先沿用当前完整数据包的保存方式，后续是否改为只提交变化，由双方共同确定。

向算法要什么

提供一份前端能直接验证的样本，说明牙号、标签对应顶点还是三角面、模型单位、坐标方向，以及边界和牙轴的含义。标签需要与解压后的模型顺序对应，否则模型能显示，分区也可能错位。

复杂重算还需要说明输入和输出。例如前端提交调整后的边界，算法是否返回新标签、新单牙网格，或者两者都返回。前端按明确结果更新显示，不能自行推断哪个字段有效。

五 哪些放前端 哪些交给算法

适合前端：模型展示、选牙、拖动控制点、位置旋转、颜色预览，以及性能允许时的局部几何计算。现有代码已在浏览器实现部分标签编辑和碰撞查询，可以作为起点。

适合进一步交给算法：重新生成正式分割网格、更复杂的边界处理、需要影像支持的真实牙根和专业分析。是否迁移现有计算，应根据模型规模、响应速度和结果要求决定。

六 前端可以按什么顺序推进

先展示：用一份约定样本完成上下颌、牙号和颜色分区，确认坐标与方向正确。

再编辑：先做通一颗牙的边界或姿态操作，区分预览变化和实际数据变化。

再保存：打通提交、保存、重新加载，确认同一颗牙的效果可以恢复；然后接入需要算法重算的功能。

最后扩展：统一两个页面对边界和姿态的读取规则，增加更多病例验证，再针对卡顿环节优化。

可以沿用这条前端路线，但要明确当前边界预览不一定更新标签，模拟牙根也不代表真实牙根。前端首先应保证操作有反馈、编辑结果说得清、保存后能恢复，再逐步补充算法精度和业务能力。